ANEXO DEL INFORME

La última transmisión

Modelos, arquitectura, matriz de trazabilidad y evidencias de trabajo

Integrantes	Rodrigo Andres Facio Lince Beltran · T00069253 Luis Gustavo Ruiz Teran · T00067451
Docente	Aisner José Marrugo Juliao
Asignatura	Ingeniería de Software · Universidad Tecnológica de Bolívar
Fecha	14 de octubre de 2026
Versión	Final (posterior a la solicitud de cambio de la hora 6: deshacer la última decisión)

1. Alcance del anexo

Este anexo complementa el informe y reúne, para la **versión final** de la solución, los cinco modelos exigidos (contexto, casos de uso, clases, secuencia y estados), el diagrama de arquitectura, la matriz de trazabilidad y las evidencias de trabajo (tablero e historial de Git). Los nombres de clases, métodos y archivos corresponden al código entregado (`motor.py`, `cli.py`, `test_motor.py`, `test_cambio_hora6.py`).

2. Modelos del software

2.1 Diagrama de contexto

El Escritor redacta y edita archivos `.json`; el Lector interactúa por consola con `cli.py`, que lee el archivo y delega las reglas en `motor.py`. No hay base de datos, red ni guardado entre ejecuciones.

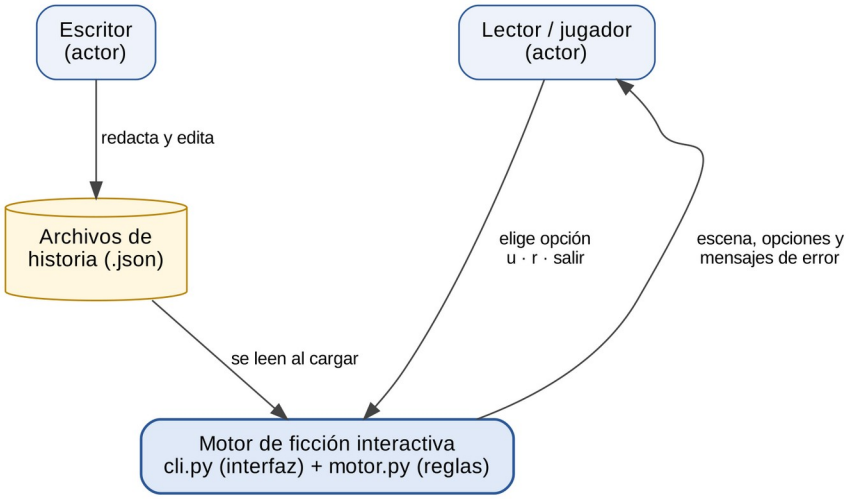


Figura 1. Diagrama de contexto del motor de ficción interactiva.

2.2 Diagrama de casos de uso

CU6 (resaltado) se incorporó con la solicitud de cambio y extiende la elección de opciones. Cargar la historia siempre incluye validar el contenido. El Escritor se relaciona con CU2 porque los mensajes de validación le indican qué corregir antes de que alguien juegue.

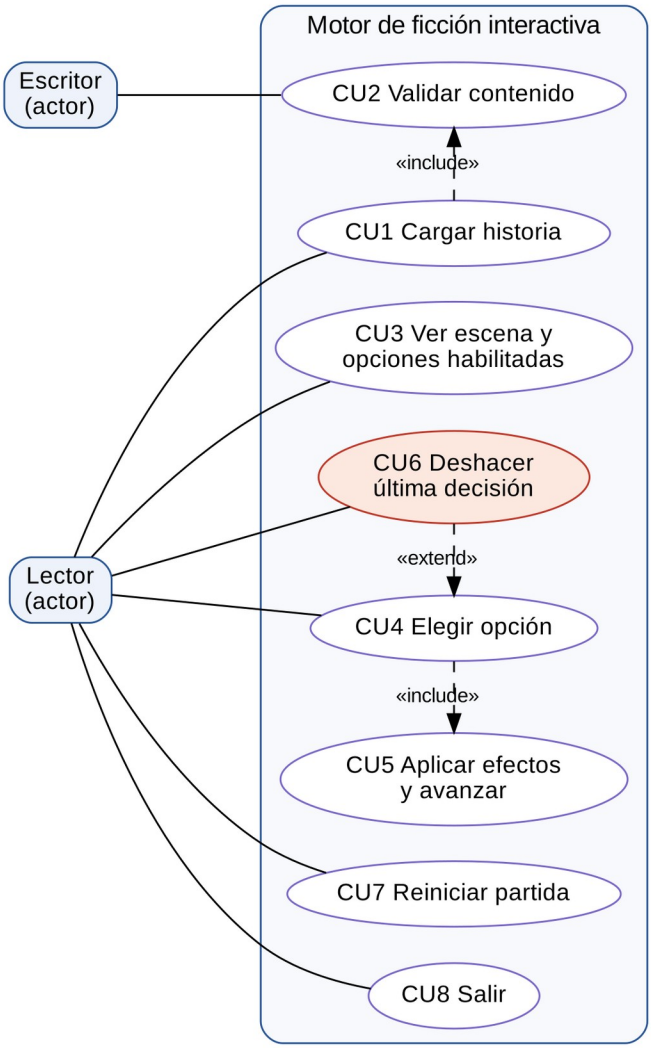


Figura 2. Casos de uso del Lector y del Escritor (CU6 corresponde al cambio de la hora 6).

2.3 Diagrama de clases del dominio

`Historia`, `Escena` y `Opcion` son estructuras de datos que se leen del JSON (diccionarios en Python). `ValidadorHistoria` y `Partida` (en `motor.py`) y las funciones de `cli.py` son el código ejecutable. El atributo `historial` de `Partida` guarda una tupla (escena anterior, copia de las variables) o `None`, y es lo que implementa el retroceso de un solo nivel.

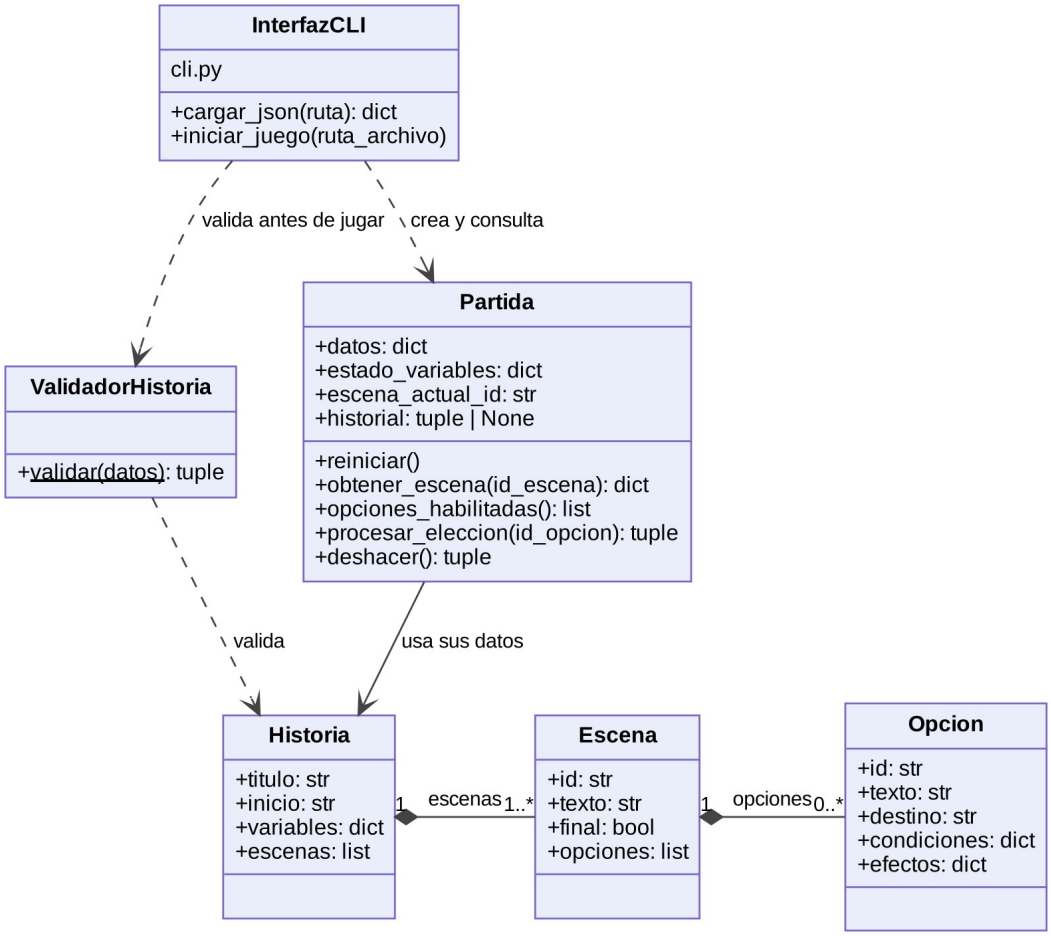


Figura 3. Diagrama de clases del dominio.

2.4 Diagramas de secuencia

La Figura 4 muestra la secuencia de elegir una opción. Si la opción no está habilitada, la sesión no cambia ni se modifica el historial. La Figura 5 muestra la secuencia de deshacer, incorporada con el cambio.

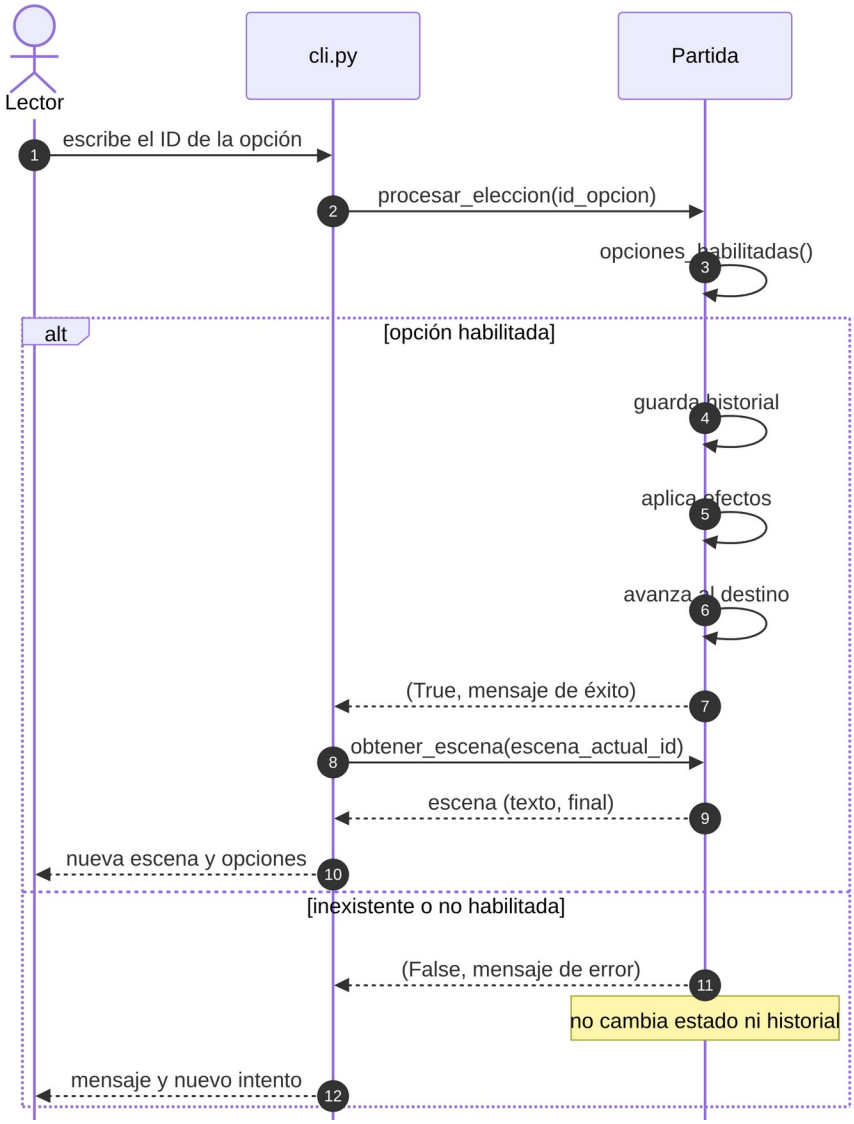


Figura 4. Secuencia de elegir una opción (caso válido y caso inválido).

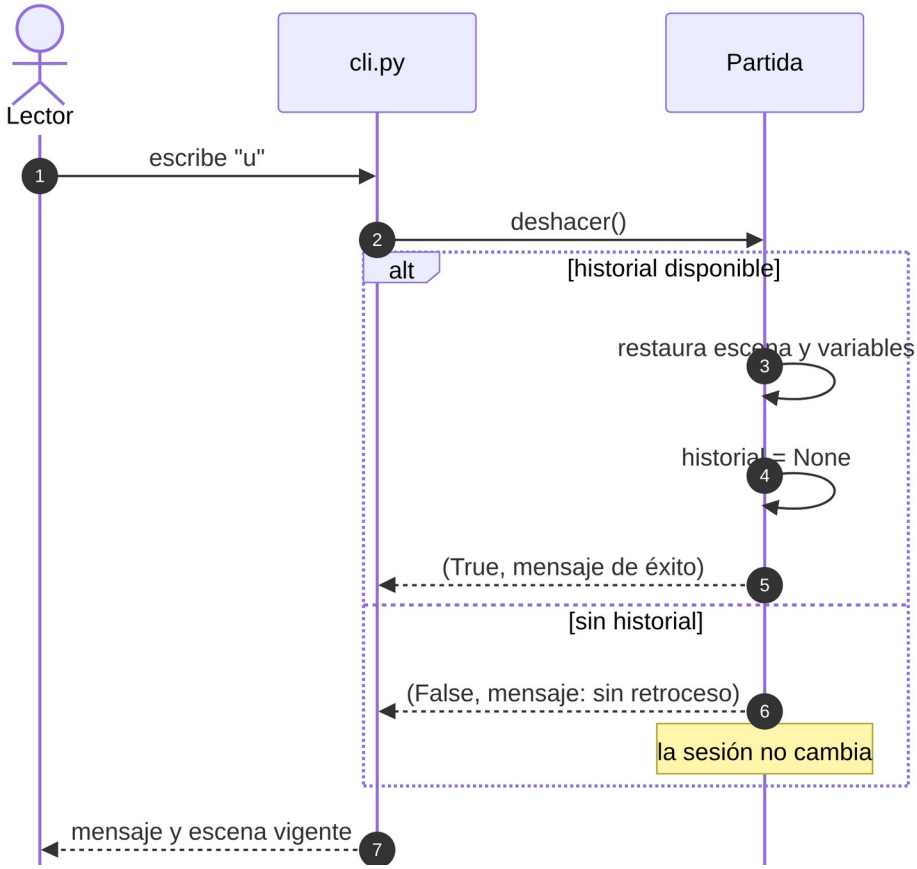


Figura 5. Secuencia de deshacer la última decisión.

2.5 Diagrama de estados de la sesión

La sesión comienza en escena activa sin retroceso. Toda elección válida guarda en `historial` la escena y las variables previas y pasa a escena activa con retroceso. Deshacer restaura ese estado y consume el historial (un solo nivel). Los finales y los bloqueos (escena no final sin opciones habilitadas) conservan el historial, por eso también se puede deshacer desde ellos. Reiniciar, desde cualquier estado, restaura la escena y las variables iniciales y elimina el historial.

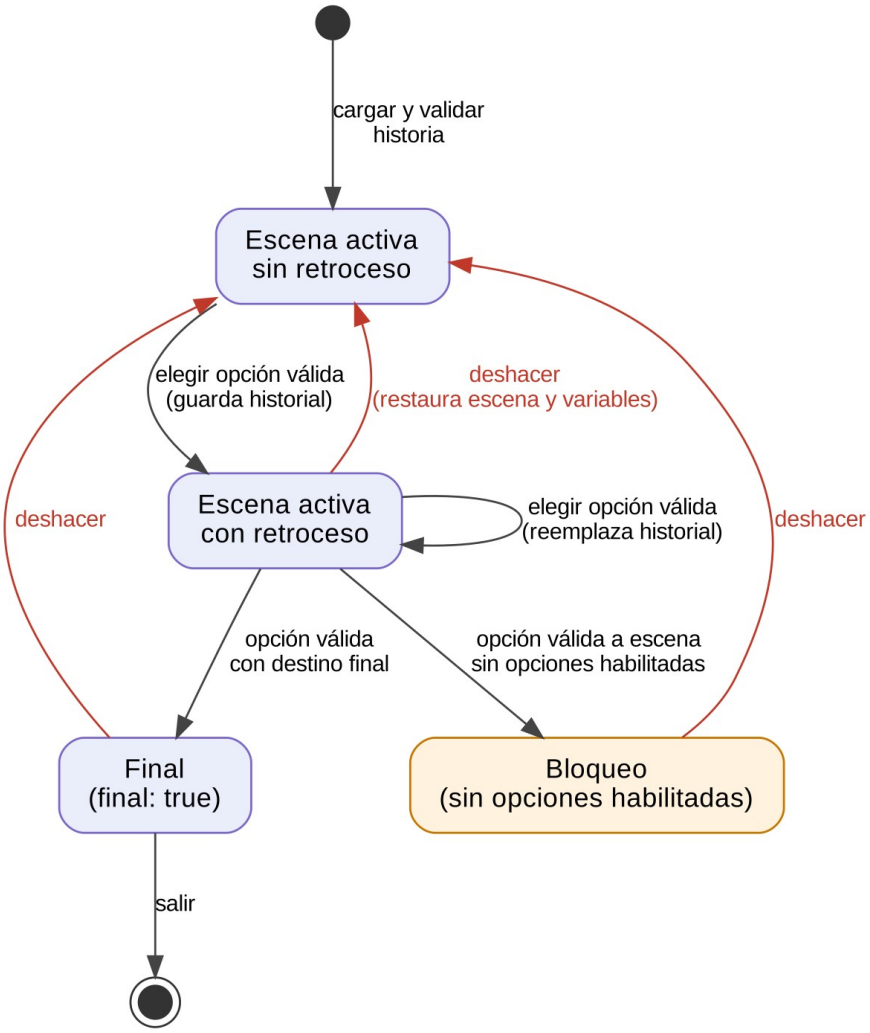


Figura 6. Estados de la sesión de juego, incluido el retroceso (en rojo, las transiciones de deshacer).

3. Arquitectura

El sistema se organiza en tres capas más la verificación. La separación entre contenido (`.json`), reglas (`motor.py`) e interfaz (`cli.py`) permite que la escritora cambie historias sin tocar código y que las reglas se prueben sin interfaz: `test_motor.py` usa `Partida` y `ValidadorHistoria` directamente. La segunda historia (`segunda_historia.json`) se carga con el mismo motor, solo cambiando la ruta.

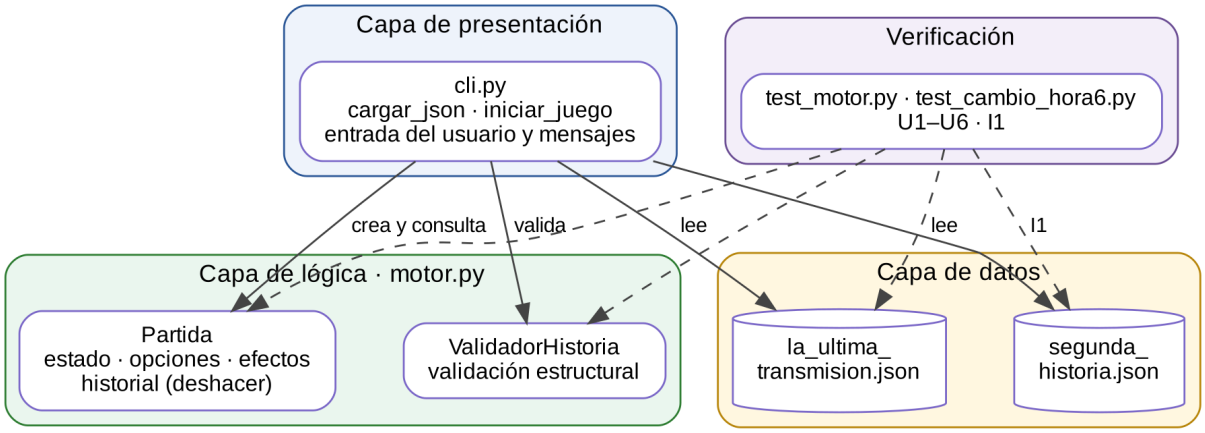


Figura 7. Diagrama de componentes por capas.

Capa	Archivo	Responsabilidad	Depende de
Presentación	`cli.py`	Leer el JSON (`cargar_json`), invocar la validación, mostrar escena y opciones, capturar comandos (opción, `u`, `r`, `salir`) e informar bloqueos y errores.	`motor.py`, `.json`
Lógica	`motor.py` › `ValidadorHistoria`	Rechazar contenido inválido antes de jugar, con un mensaje que indica la causa y el campo o ID.	Ninguna
Lógica	`motor.py` › `Partida`	Estado de la sesión, opciones habilitadas, efectos, finales, reinicio e historial de un nivel (deshacer).	Datos de la historia
Datos	`la_ultima_transmision.json`, `segunda_historia.json`	Contenido narrativo y reglas booleanas, independientes del código.	Ninguna
Verificación	`test_motor.py`, `test_cambio_hora6.py`	Pruebas unitarias U1 a U6 e integración I1, y pruebas del cambio.	`motor.py`, `.json`

4. Requisito del cambio de la hora 6 (RF-07)

Identificador	**RF-07** · Deshacer la última decisión
Origen	Solicitud de cambio de la hora 6. Reemplaza el acuerdo provisional de no implementar retroceso.
Descripción	El lector puede deshacer su última elección válida. Se restauran exactamente la escena y todas las variables anteriores a esa elección. El retroceso es de un solo nivel: tras deshacer no hay otro hasta realizar una nueva elección válida. No se exige rehacer.
Artefactos afectados	`motor.py` (`Partida.historial`, `procesar_eleccion`, `deshacer`), `cli.py` (comando u), `test_motor.py` (U6), `test_cambio_hora6.py`, supuestos del informe (flujo unidireccional) y limitaciones del README.

Criterios de aceptación de RF-07

Criterio	Descripción	Verificado con
CA-07.1	Desde E2, investigar lleva a E3 con `conoce_origen` verdadero; deshacer devuelve a E2 con `conoce_origen` falso.	`test_cambio_hora6.py` › `test_U6_ejemplo_del_enunciado`; `test_motor.py` › `test_U6…`
CA-07.2	Un segundo deshacer consecutivo no cambia la sesión (un solo nivel).	`test_motor.py` › `test_U6…`; `test_U6_ejemplo_del_enunciado`
CA-07.3	Tras deshacer, continuar sin investigar muestra en E3 «activar sin verificar» y «cortar comunicación»; la nueva elección vuelve a habilitar el retroceso.	`test_U6_ejemplo_del_enunciado`; `test_U6_variante_nueva_eleccion_rehabilita_retroceso`
CA-07.4	Al comenzar no hay retroceso disponible; reiniciar elimina el historial y restaura el estado inicial.	`test_U6_al_comenzar_no_hay_retroceso`; `test_U5_extension_reiniciar_elimina_historial`; `test_motor.py` › `test_U5…`
CA-07.5	Una opción inexistente o no habilitada no modifica la sesión ni sustituye el historial de la última elección válida.	`test_U6_variante_entrada_invalida_preserva_historial`; `test_motor.py` › `test_U3…`
CA-07.6	Desde E4, E5 o E6 se puede deshacer y se regresa a E3 con los valores previos a esa elección.	`test_I1_extension_deshacer_desde_cada_final`

5. Matriz de trazabilidad

Formato: `archivo › símbolo`. `test_cambio_hora6.py` complementa a `test_motor.py` con las extensiones del cambio (U5, U6 e I1).

5.1 Requisitos funcionales

ID	Necesidad	Requisito	Modelos	Componente / ruta	Evidencia de verificación	Versión
RF-01	N01	Cargar el JSON e inicializar la partida con `inicio` y `variables`.	Fig. 1, 2 (CU1), 3	`cli.py` › `cargar_json`, `iniciar_juego`; `motor.py` › `Partida.__init__`, `reiniciar`; `la_ultima_transmision.json`, `segunda_historia.json`	`test_motor.py` › `test_I1_integracion_segunda_historia` (archivo real, escena inicial «celda»); ejecución de `python cli.py <archivo>.json`	Inicial
RF-02	N02	Mostrar solo las opciones cuyas condiciones se cumplen.	Fig. 2 (CU3), 3, 4	`motor.py` › `Partida.opciones_habilitadas`	`test_motor.py` › `test_U1_condiciones_habilitan_opciones`; `test_I1_integracion_segunda_historia`	Inicial
RF-03	N03	Una elección válida aplica los efectos y avanza al destino; una entrada inválida no altera la partida.	Fig. 2 (CU4, CU5), 4, 6	`motor.py` › `Partida.procesar_eleccion`	`test_motor.py` › `test_U2_eleccion_valida_aplica_efectos`, `test_U3_opcion_invalida_no_modifica_estado`; `test_I1_integracion_segunda_historia`	Inicial
RF-04	N04	Conservar las variables modificadas durante la partida.	Fig. 3, 6	`motor.py` › `Partida.estado_variables`, `procesar_eleccion`	`test_motor.py` › `test_U2_eleccion_valida_aplica_efectos`; `test_I1_integracion_segunda_historia`	Inicial
RF-05	N05	Reconocer el final de la partida y permitir reiniciar sin conservar información anterior.	Fig. 2 (CU7), 6	`cli.py` › `iniciar_juego` (rama `escena['final']`); `motor.py` › `Partida.reiniciar`	`test_motor.py` › `test_U5_reiniciar_restaura_estado`; `test_I1_integracion_segunda_historia` (llega al final «capturado»)	Inicial
RF-06	N06	Rechazar antes de jugar los archivos inválidos, indicando la causa y el campo o ID.	Fig. 2 (CU2), 3	`motor.py` › `ValidadorHistoria.validar`; `cli.py` › `iniciar_juego`	`test_motor.py` › `test_U4_errores_de_contenido_obligatorios` (destino inexistente; escena final con opciones); README › Manejo de errores	Inicial
RF-07	Cambio hora 6	Deshacer la última decisión (un nivel), restaurando escena y variables, también desde un final.	Fig. 2 (CU6), 3, 5, 6	`motor.py` › `Partida.historial`, `procesar_eleccion` (guarda), `Partida.deshacer`; `cli.py` › comando `u` (escena, final y bloqueo)	`test_motor.py` › `test_U6_deshacer_restaura_estado_y_consume_reproceso`; `test_cambio_hora6.py` › U5 extendida, variantes de U6, I1 extendida (CA-07.1 a CA-07.6); commits `3bf1fcb`, `8b14283`, `d3ff396`	Cambio

5.2 Requisitos no funcionales

ID	Atributo	Medida verificable	Procedimiento	Componente	Evidencia
RNF-01	Modificabilidad	Una historia nueva se ejecuta modificando 0 archivos de código: solo se indica otra ruta `.json`.	Ejecutar `python cli.py segunda_historia.json` y `python cli.py la_ultima_transmision.json` con los mismos `motor.py` y `cli.py`; comprobar con `git diff` que ningún `.py` cambia entre ambas; correr I1.	`cli.py` (ruta como argumento); `motor.py` (sin identificadores de escena); `.json`	`test_motor.py` › `test_I1_integracion_segunda_historia`
RNF-02	Confiabilidad	El 100 % de las entradas inválidas probadas (vacía, inexistente, no habilitada, con mayúsculas o espacios) deja escena, variables e historial sin cambios y no interrumpe la consola.	Invocar `procesar_eleccion` con 8 entradas inválidas y comparar la instantánea de la sesión antes y después; repetir en la consola.	`motor.py` › `Partida.procesar_eleccion`; `cli.py` › mensaje «Intenta de nuevo»	`test_motor.py` › `test_U3…`; `test_cambio_hora6.py` › `test_RNF02_entradas_invalidas_no_alteran_la_sesion`
RNF-03	Claridad de errores	Cada error de contenido probado produce un mensaje que nombra la causa y el campo o ID involucrado, cuando es identificable.	Inyectar errores en una copia del contenido (destino inexistente, escena final con opciones) y comprobar el texto de `ValidadorHistoria.validar`; reproducirlo en consola editando `segunda_historia.json`.	`motor.py` › `ValidadorHistoria.validar`; `cli.py` › `iniciar_juego`	`test_motor.py` › `test_U4_errores_de_contenido_obligatorios`

5.3 Cobertura de pruebas

Prueba	Archivo › función	Qué verifica	Requisitos
U1	`test_motor.py` › `test_U1_condiciones_habilitan_opciones`	Solo las opciones con condiciones cumplidas quedan habilitadas.	RF-02
U2	`test_motor.py` › `test_U2_eleccion_valida_aplica_efectos`	Se aplican los efectos y se avanza al destino.	RF-03, RF-04
U3	`test_motor.py` › `test_U3_opcion_invalida_no_modifica_estado`	Una opción no habilitada deja el estado intacto.	RF-03, RNF-02
U4	`test_motor.py` › `test_U4_errores_de_contenido_obligatorios`	Diagnóstico controlado ante destino inexistente y escena final con opciones.	RF-06, RNF-03
U5	`test_motor.py` › `test_U5_reiniciar_restaura_estado`; `test_cambio_hora6.py` › `test_U5_extension_reiniciar_elimina_historial`	Reiniciar restaura escena y variables y elimina el historial.	RF-05, RF-07
U6	`test_motor.py` › `test_U6_deshacer_restaura_estado_y_consume_retroceso`; `test_cambio_hora6.py` › `test_U6_ejemplo_del_enunciado`, `test_U6_al_comenzar_no_hay_retroceso`, `test_U6_variante_entrada_invalida_preserva_historial`, `test_U6_variante_nueva_eleccion_rehabilita_retroceso`	Deshacer restaura escena y variables y consume el retroceso; la entrada inválida preserva el historial; una nueva elección vuelve a habilitarlo.	RF-07
I1	`test_motor.py` › `test_I1_integracion_segunda_historia`; `test_cambio_hora6.py` › `test_I1_extension_deshacer_desde_cada_final`	Carga de archivo real y decisiones hasta un final; deshacer desde E4, E5 y E6 regresa a E3.	RF-01 a RF-05, RF-07, RNF-01
RNF-02	`test_cambio_hora6.py` › `test_RNF02_entradas_invalidas_no_alteran_la_sesion`	Ocho entradas inválidas no alteran la sesión.	RNF-02

Última ejecución (14 de octubre de 2026): `python -m unittest test_motor test_cambio_hora6 -v` → 14 pruebas ejecutadas, 0 fallos, 0 errores (resultado OK).

6. Evidencia de trabajo

6.1 Tablero de seguimiento (Trello)

Las tres capturas muestran la evolución del tablero: al inicio había 3 tareas por hacer, 3 en curso y 1 hecha; en la fase intermedia quedaban 2 en curso y 5 hechas; al cierre las 7 tareas están en «Hecho».

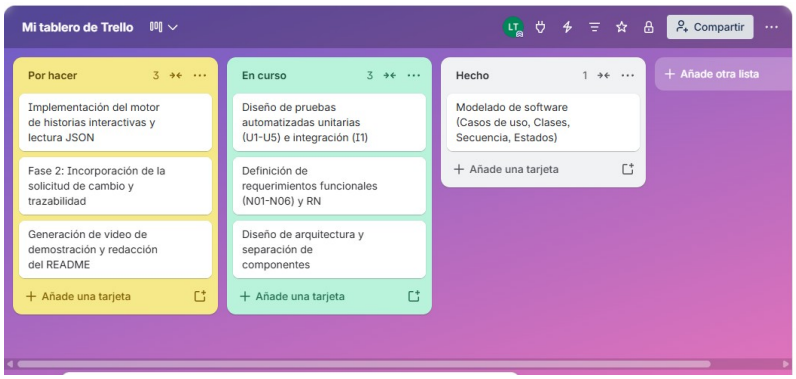


Figura 8. Tablero inicial: 3 por hacer · 3 en curso · 1 hecho.

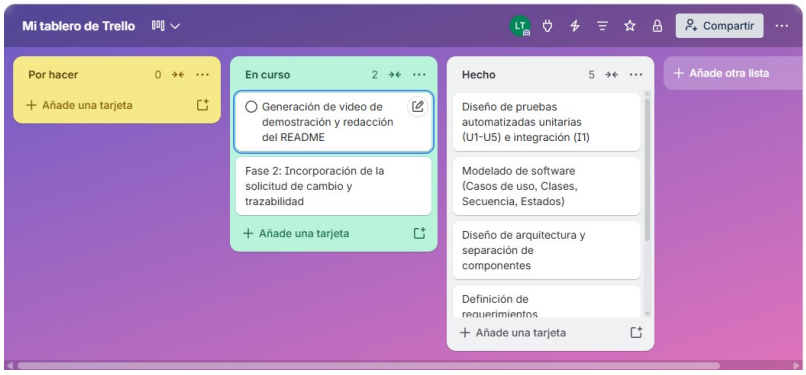


Figura 9. Tablero intermedio: 0 por hacer · 2 en curso · 5 hechos.

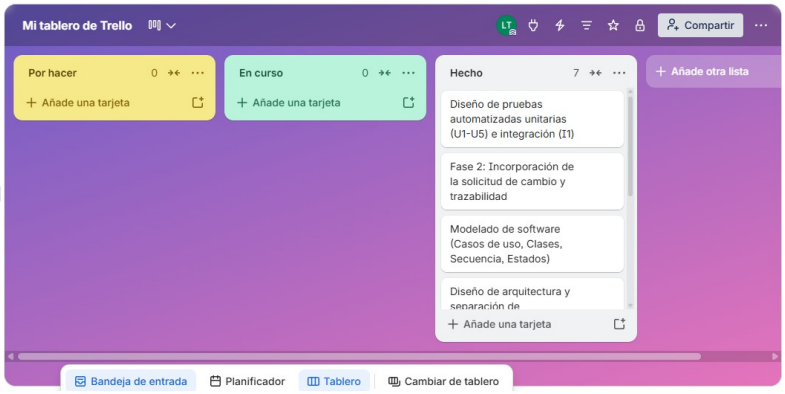


Figura 10. Tablero final: 0 por hacer · 0 en curso · 7 hechos.

6.2 Historial de Git y versiones antes y después del cambio

Versión	Commit	Mensaje	Autor
Entrega de la primera parte	`88df4bc`	Primera parte del parcial (6 horas)	Rodrigo Facio Lince
Anterior al cambio (último commit antes de la solicitud)	`b9c3375`	Update README.md	LGustavo09
Cambio: lógica y deshacer	`3bf1fcb`	feat: implementar validacion de contenido, logica de partida y mecanica de deshacer segun solicitud de cambio	LGustavo09
Cambio: interfaz	`8b14283`	feat: actualizar cli para incluir opcion de deshacer la ultima decision	LGustavo09
Posterior al cambio	`d3ff396`	test: agregar la prueba unitaria U6 para verificar el mecanismo de deshacer	LGustavo09

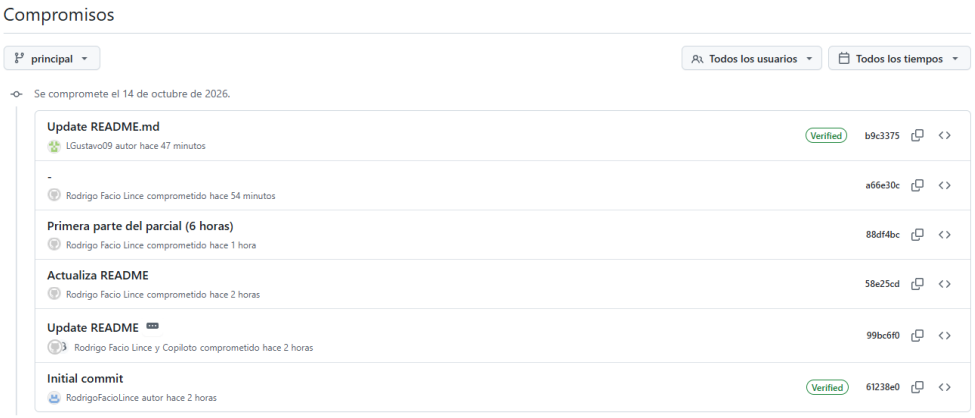


Figura 11. Historial de commits antes de la solicitud de cambio (rama principal).

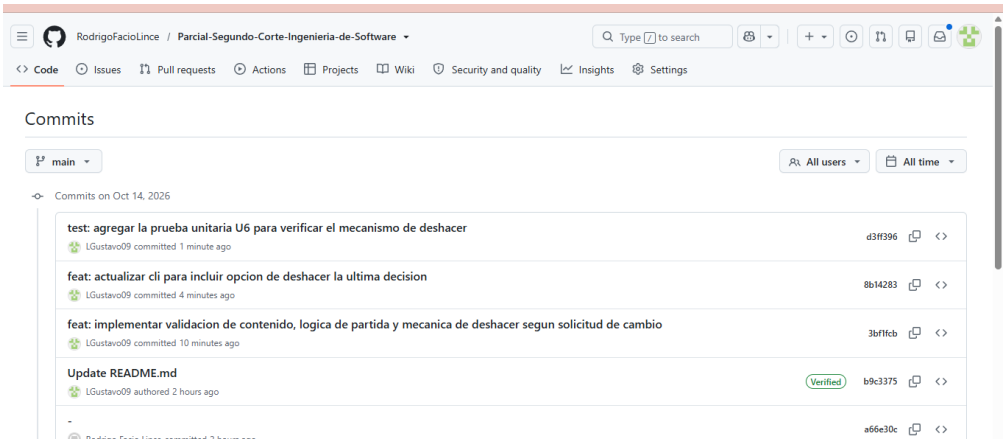


Figura 12. Historial de commits después del cambio (rama main): lógica, interfaz y prueba U6.